Latest updates: https://dl.acm.org/doi/10.1145/602259.602266

ARTICLE

R-trees: a dynamic index structure for spatial searching

ANTONIN GUTTMAN, University of California, Berkeley, Berkeley, CA, United States

Open Access Support provided by:

University of California, Berkeley

PDF Download
602259.602266.pdf
24 March 2026
Total Citations: 5754
Total Downloads: 23525

Published: 01 June 1984

Citation in BibTeX format

Conference Sponsors:
SIGMOD

SIGMOD '84: Proceedings of the 1984 ACM SIGMOD international conference on Management of data (June 1984)
https://doi.org/10.1145/602259.602266
ISBN: 0897911288

R-TREES: A DYNAMIC INDEX STRUCTURE FOR SPATIAL SEARCHING

Antonin Guttman
University of California
Berkeley

Abstract

In order to handle spatial data efficiently, as required in computer aided design and geo-data applications, a database system needs an index mechanism that will help it retrieve data items quickly according to their spatial locations. However, traditional indexing methods are not well suited to data objects of non-zero size located in multi-dimensional spaces. In this paper we describe a dynamic index structure called an R-tree which meets this need, and give algorithms for searching and updating it. We present the results of a series of tests which indicate that the structure performs well, and conclude that it is useful for current database systems in spatial applications.

1. Introduction

Spatial data objects often cover areas in multi-dimensional spaces and are not well represented by point locations. For example, map objects like counties, census tracts etc. occupy regions of non-zero size in two dimensions. A common operation on spatial data is a search for all objects in an area, for example to find all counties that have land within 20 miles of a particular point. This kind of spatial search occurs frequently in computer aided design (CAD) and geo-data applications, and therefore it is important to be able to retrieve objects efficiently according to their spatial location.

This research was sponsored by National Science Foundation grant ECS-8300463 and Air Force Office of Scientific Research grant AFOSR-83-0254.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1984 ACM 0-89791-128-8/84/006/0047 $00.75

An index based on objects' spatial locations is desirable, but classical one-dimensional database indexing structures are not appropriate to multi-dimensional spatial searching. Structures based on exact matching of values, such as hash tables, are not useful because a range search is required. Structures using one-dimensional ordering of key values, such as B-trees and ISAM indexes, do not work because the search space is multi-dimensional.

A number of structures have been proposed for handling multi-dimensional point data, and a survey of methods can be found in [5]. Cell methods [4, 8, 16] are not good for dynamic structures because the cell boundaries must be decided in advance. Quad trees [7] and k-d trees [3] do not take paging of secondary memory into account. K-D-B trees [13] are designed for paged memory but are useful only for point data. The use of index intervals has been suggested in [15], but this method cannot be used in multiple dimensions. Corner stitching [12] is an example of a structure for two-dimensional spatial searching suitable for data objects of non-zero size, but it assumes homogeneous primary memory and is not efficient for random searches in very large collections of data. Grid files [10] handle non-point data by mapping each object to a point in a

higher-dimensional space. In this paper we describe an alternative structure called an R-tree which represents data objects by intervals in several dimensions.

Section 2 outlines the structure of an R-tree and Section 3 gives algorithms for searching, inserting, deleting, and updating. Results of R-tree index performance tests are presented in Section 4. Section 5 contains a summary of our conclusions.

2. R-Tree Index Structure

An R-tree is a height-balanced tree similar to a B-tree [2,6] with index records in its leaf nodes containing pointers to data objects. Nodes correspond to disk pages if the index is disk-resident, and the structure is designed so that a spatial search requires visiting only a small number of nodes. The index is completely dynamic; inserts and deletes can be intermixed with searches and no periodic reorganization is required.

A spatial database consists of a collection of tuples representing spatial objects, and each tuple has a unique identifier which can be used to retrieve it. Leaf nodes in an R-tree contain index record entries of the form

$$(I,\ \textit{tuple-identifier})$$

where *tuple-identifier* refers to a tuple in the database and I is an n-dimensional rectangle which is the bounding box of the spatial object indexed:

$$I = (I_0, I_1, \ldots, I_{n-1})$$

Here n is the number of dimensions and I_i is a closed bounded interval $[a, b]$ describing the extent of the object along dimension i. Alternatively I_i may have one or both endpoints equal to infinity, indicating that the object extends outward indefinitely. Non-leaf nodes contain entries of the form

$$(I,\ \textit{child-pointer})$$

where *child-pointer* is the address of a lower node in the R-tree and I covers all rectangles in the lower node's entries.

Let M be the maximum number of entries that will fit in one node and let $m \le \frac{M}{2}$ be a parameter specifying the minimum number of entries in a node. An R-tree satisfies the following properties:

(1) Every leaf node contains between m and M index records unless it is the root.

(2) For each index record $(I,\ \textit{tuple-identifier})$ in a leaf node, I is the smallest rectangle that spatially contains the n-dimensional data object represented by the indicated tuple.

(3) Every non-leaf node has between m and M children unless it is the root.

(4) For each entry $(I,\ \textit{child-pointer})$ in a non-leaf node, I is the smallest rectangle that spatially contains the rectangles in the child node.

(5) The root node has at least two children unless it is a leaf.

(6) All leaves appear on the same level.

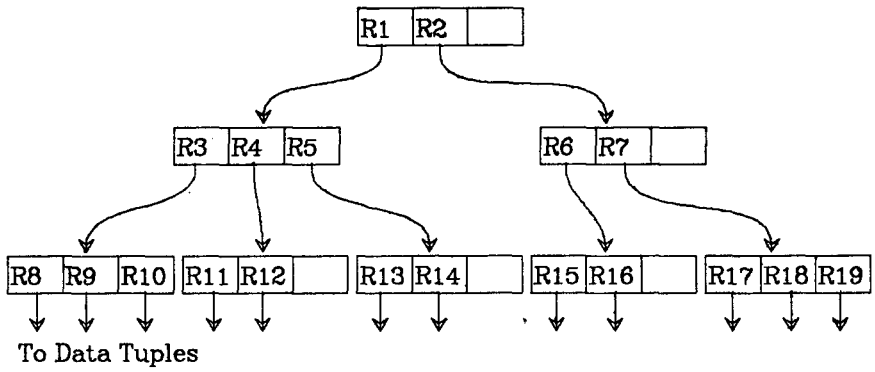
Figure 2.1a and 2.1b show the structure of an R-tree and illustrate the containment and overlapping relationships that can exist between its rectangles.

The height of an R-tree containing N index records is at most $\lceil \log_m N \rceil - 1$, because the branching factor of each node is at least m. The maximum number of nodes is $\left\lceil \frac{N}{m} \right\rceil + \left\lceil \frac{N}{m^2} \right\rceil + \cdots + 1$. Worst-case space utilization for all nodes except the root is $\frac{m}{M}$. Nodes will tend to have more than m entries, and this will decrease tree height and improve space utilization. If nodes have more than 3 or 4 entries the tree is very wide, and almost all the space is used for leaf nodes containing index records. The parameter m can be varied as part of performance tuning, and different values are tested experimentally in Section 4.

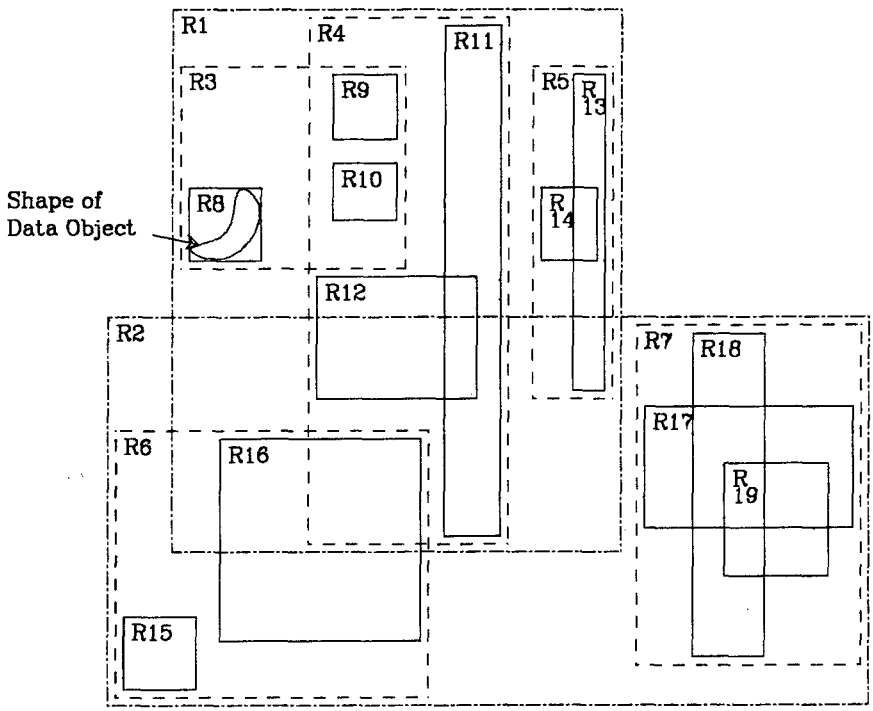
3. Searching and Updating

3.1. Searching

The search algorithm descends the tree from the root in a manner similar to a B-tree. However, more than one subtree under a node visited may need to be searched, hence it is not possible to guarantee good worst-case performance. Nevertheless with most kinds of data the update algorithms will maintain the tree in a form that allows the search algorithm to eliminate irrelevant regions of the indexed space, and examine only data near the



(a)



(b)

Figure 3.1

search area.

In the following we denote the rectangle part of an index entry E by $E.I$, and the *tuple–identifier* or *child–pointer* part by $E.p$.

Algorithm **Search.** Given an R-tree whose root node is T, find all index records whose rectangles overlap a search rectangle S.

S1. [Search subtrees.] If T is not a leaf, check each entry E to determine whether $E.I$ overlaps S. For all overlapping entries, invoke **Search** on the tree whose root node is pointed to by $E.p$.

S2. [Search leaf node.] If T is a leaf, check all entries E to determine whether $E.I$ overlaps S. If so, E is a qualifying record.

3.2. Insertion

Inserting index records for new data tuples is similar to insertion in a B-tree in that new index records are added to the leaves, nodes that overflow are split, and splits propagate up the tree.

Algorithm **Insert.** Insert a new index entry E into an R-tree.

I1. [Find position for new record.] Invoke **ChooseLeaf** to select a leaf node L in which to place E.

I2. [Add record to leaf node.] If L has room for another entry, install E. Otherwise invoke **SplitNode** to obtain L and LL containing E and all the old entries of L.

I3. [Propagate changes upward.] Invoke **AdjustTree** on L, also passing LL if a split was performed.

I4. [Grow tree taller.] If node split propagation caused the root to split, create a new root whose children are the two resulting nodes.

Algorithm **ChooseLeaf**. Select a leaf node in which to place a new index entry E.

CL1. [Initialize.] Set N to be the root node.

CL2. [Leaf check.] If N is a leaf, return N.

CL3. [Choose subtree.] If N is not a leaf, let F be the entry in N whose rectangle $F.I$ needs least enlargement to include $E.I$. Resolve ties by choosing the entry with the rectangle of smallest area.

CL4. [Descend until a leaf is reached.] Set N to be the child node pointed to by $F.p$ and repeat from CL2.

Algorithm **AdjustTree**. Ascend from a leaf node L to the root, adjusting covering rectangles and propagating node splits as necessary.

AT1. [Initialize.] Set $N=L$. If L was split previously, set NN to be the resulting second node.

AT2. [Check if done.] If N is the root, stop.

AT3. [Adjust covering rectangle in parent entry.] Let P be the parent node of N, and let E_N be N's entry in P. Adjust $E_N.I$ so that it tightly encloses all entry rectangles in N.

AT4. [Propagate node split upward.] If N has a partner NN resulting from an earlier split, create a new entry E_{NN} with $E_{NN}.p$ pointing to NN and $E_{NN}.I$ enclosing all rectangles in NN. Add E_{NN} to P if there is room. Otherwise, invoke **SplitNode** to produce P and PP containing E_{NN} and all P's old entries.

AT5. [Move up to next level.] Set $N=P$ and set $NN=PP$ if a split occurred. Repeat from AT2.

Algorithm **SplitNode** is described in Section 3.5.

3.3. Deletion

Algorithm **Delete**. Remove index record E from an R-tree.

D1. [Find node containing record.] Invoke **FindLeaf** to locate the leaf node L containing E. Stop if the record was not found.

D2. [Delete record.] Remove E from L.

D3. [Propagate changes.] Invoke **CondenseTree**, passing L.

D4. [Shorten tree.] If the root node has only one child after the tree has been adjusted, make the child the new root.

Algorithm **FindLeaf**. Given an R-tree whose root node is T, find the leaf node containing the index entry E.

FL1. [Search subtrees.] If T is not a leaf, check each entry F in T to determine if $F.I$ overlaps $E.I$. For each such entry invoke **FindLeaf** on the tree whose root is pointed to by $F.p$ until E is found or all entries have been checked.

FL2. [Search leaf node for record.] If T is a leaf, check each entry to see if it matches E. If E is found return T.

Algorithm **CondenseTree**. Given a leaf node L from which an entry has been deleted, eliminate the node if it has too few entries and relocate its entries. Propagate node elimination upward as necessary. Adjust all covering rectangles on the path to the root, making them smaller if possible.

CT1. [Initialize.] Set $N=L$. Set Q, the set of eliminated nodes, to be empty.

CT2. [Find parent entry.] If N is the root, go to CT6. Otherwise let P be the parent of N, and let E_N be N's entry in P.

CT3. [Eliminate under-full node.] If N has fewer than m entries, delete E_N from P and add N to set Q.

CT4. [Adjust covering rectangle.] If N has not been eliminated, adjust $E_N.I$ to tightly contain all entries in N.

CT5. [Move up one level in tree.] Set $N=P$ and repeat from CT2.

CT6. [Re-insert orphaned entries.] Re-insert all entries of nodes in set Q. Entries from eliminated leaf nodes are re-inserted in tree leaves as described in Algorithm **Insert**, but entries from higher-level nodes must be placed higher in the tree, so that leaves of their dependent subtrees will be on the same level as leaves of the main tree.

The procedure outlined above for disposing of under-full nodes differs from the corresponding operation on a B-tree, in which two or more adjacent nodes are merged. A B-tree-like approach is possible for R-trees, although there is no adjacency in the B-tree sense: an under-full node can be merged with whichever sibling will have its area increased least, or the orphaned entries can be distributed among sibling nodes. Either method can cause nodes to be split. We chose re-insertion instead for two reasons: first, it accomplishes the same thing and is easier to implement because the **Insert** routine can be used. Efficiency should be comparable because pages needed during re-insertion usually will be the same ones visited during the preceding search and will already be in memory. The second reason is that re-insertion incrementally refines the spatial structure of the tree, and prevents gradual deterioration that might occur if each entry were located permanently under the same parent node.

3.4. Updates and Other Operations

If a data tuple is updated so that its covering rectangle is changed, its index record must be deleted, updated, and then re-inserted, so that it will find its way to the right place in the tree.

Other kinds of searches besides the one described above may be useful, for example to find all data objects completely contained in a search area, or all objects that contain a search area. These operations can be implemented by straightforward variations on the algorithm given. A search for a specific entry whose identity is known beforehand is required by the deletion algorithm and is implemented by Algorithm **FindLeaf**. Variants of range deletion, in which index entries for all data objects in a particular area are removed, are also well supported by R-trees.

3.5. Node Splitting

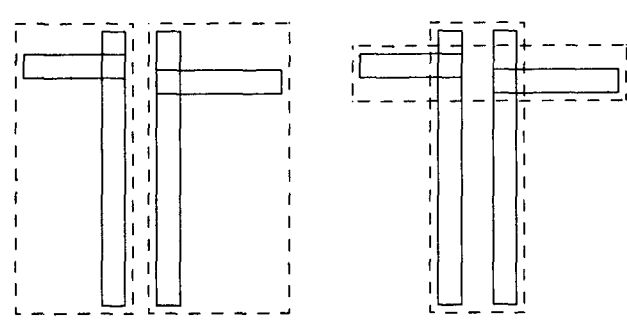
In order to add a new entry to a full node containing M entries, it is necessary to divide the collection of $M+1$ entries between two nodes. The division should be done in a way that makes it as unlikely as possible that both new nodes will need to be examined on subsequent searches. Since the decision whether to visit a node depends on whether its covering rectangle overlaps the search area, the total area of the two covering rectangles after a split should be minimized. Figure 3.1 illustrates this point. The area of the covering rectangles in the "bad split" case is much larger than in the "good split" case.

The same criterion was used in procedure **ChooseLeaf** to decide where to insert a new index entry: at each level in the tree, the subtree chosen was the one whose covering rectangle would have to be enlarged least.

We now turn to algorithms for partitioning the set of $M+1$ entries into two groups, one for each new node.

3.5.1. Exhaustive Algorithm

The most straightforward way to find the minimum area node split is to generate all possible groupings and choose the best. However, the number of possibilities is approximately 2^{M-1} and a reasonable value



Bad split Good split

Figure 3.1

of M is 50*, so the number of possible splits is very large. We implemented a modified form of the exhaustive algorithm to use as a standard for comparison with other algorithms, but it was too slow to use with large node sizes.

3.5.2. A Quadratic-Cost Algorithm

This algorithm attempts to find a small-area split, but is not guaranteed to find one with the smallest area possible. The cost is quadratic in M and linear in the number of dimensions. The algorithm picks two of the $M+1$ entries to be the first elements of the two new groups by choosing the pair that would waste the most area if both were put in the same group, i.e. the area of a rectangle covering both entries, minus the areas of the entries themselves, would be greatest. The remaining entries are then assigned to groups one at a time. At each step the area expansion required to add each remaining entry to each group is calculated, and the entry assigned is the one showing the greatest difference between the two groups.

Algorithm **Quadratic Split**. Divide a set of $M+1$ index entries into two groups.

QS1. [Pick first entry for each group.] Apply Algorithm **PickSeeds** to choose two entries to be the first elements of the groups. Assign each to a group.

QS2. [Check if done.] If all entries have been assigned, stop. If one group has so few entries that all the rest must be assigned to it in order for it to have the minimum number m, assign them and stop.

QS3. [Select entry to assign.] Invoke Algorithm **PickNext** to choose the next entry to assign. Add it to the group whose covering rectangle will have to be enlarged least to accommodate it. Resolve ties by adding the entry to the group with smaller area, then to the one with fewer entries, then to either. Repeat from QS2.

*A two dimensional rectangle can be represented by four numbers of four bytes each. If a pointer also takes four bytes, each entry requires 20 bytes. A page of 1024 bytes will hold about 50 entries.

Algorithm **PickSeeds**. Select two entries to be the first elements of the groups.

PS1. [Calculate inefficiency of grouping entries together.] For each pair of entries E_1 and E_2, compose a rectangle J including $E_1.I$ and $E_2.I$. Calculate $d =$ area(J) - area($E_1.I$) - area($E_2.I$).

PS2. [Choose the most wasteful pair.] Choose the pair with the largest d.

Algorithm **PickNext**. Select one remaining entry for classification in a group.

PN1. [Determine cost of putting each entry in each group.] For each entry E not yet in a group, calculate $d_1 =$ the area increase required in the covering rectangle of Group 1 to include $E.I$. Calculate d_2 similarly for Group 2.

PN2. [Find entry with greatest preference for one group.] Choose any entry with the maximum difference between d_1 and d_2.

3.5.3. A Linear-Cost Algorithm

This algorithm is linear in M and in the number of dimensions. **Linear Split** is identical to **Quadratic Split** but uses a different version of **PickSeeds**. **PickNext** simply chooses any of the remaining entries.

Algorithm **LinearPickSeeds**. Select two entries to be the first elements of the groups.

LPS1. [Find extreme rectangles along all dimensions.] Along each dimension, find the entry whose rectangle has the highest low side, and the one with the lowest high side. Record the separation.

LPS2. [Adjust for shape of the rectangle cluster.] Normalize the separations by dividing by the width of the entire set along the corresponding dimension.

LPS3. [Select the most extreme pair.] Choose the pair with the greatest normalized separation along any dimension.

4. Performance Tests

We implemented R-trees in C under Unix on a Vax 11/780 computer, and used our implementation in a series of performance tests whose purpose was to verify the practicality of the structure, to choose values for M and m, and to evaluate different node-splitting algorithms. This section presents the results.

Five page sizes were tested, corresponding to different values of M:

Bytes per Page	Max Entries per Page (M)
128	6
256	12
512	25
1024	50
2048	102

Values tested for m, the minimum number of entries in a node, were $M/2$, $M/3$, and 2. The three node split algorithms described earlier were implemented in different versions of the program. All our tests used two-dimensional data, although the structure and algorithms work for any number of dimensions.

During the first part of each test run the program read geometry data from files and constructed an index tree, beginning with an empty tree and calling *Insert* with each new index record. Insert performance was measured for the last 10% of the records, when the tree was nearly its final size. During the second phase the program called the function *Search* with search rectangles made up using random numbers. 100 searches were performed per test run, each retrieving about 5% of the data. Finally the program read the input files a second time and called the function *Delete* to remove the index record for every tenth data item, so that measurements were taken for scattered deletion of 10% of the index records. The tests were done using Very Large Scale Integrated circuit (VLSI) layout data from the RISC-II computer chip [11]. The circuit cell CENTRAL, containing 1057 rectangles, was used in the tests and is shown in Figure 4.1.

Figure 4.2 shows the cost in CPU time for inserting the last 10% of the records as a function of page size. The exhaustive algorithm, whose cost increases exponentially with page size, is seen to be very slow for larger page sizes. The linear algorithm is fastest, as expected. With this algorithm

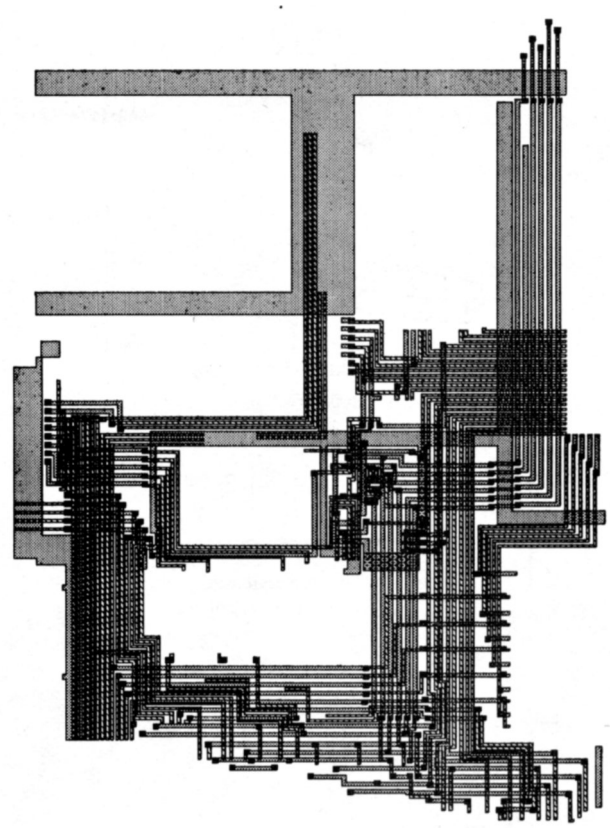


Figure 4.1
Circuit cell CENTRAL (1057 rectangles).

CPU time hardly increased with page size at all, which suggests that node splitting was responsible for only a small part of the cost of inserting records. The decreased cost of insertion with a stricter node balance requirement reflects the fact that when one group becomes too full, all split algorithms simply put the remaining elements in the other group without further comparisons.

The cost of deleting an item from the index, shown in Figure 4.3, is strongly affected by the minimum node fill requirement. When nodes become under-full, their entries must be re-inserted, and reinsertion sometimes causes nodes to split. Stricter fill requirements cause nodes to become under-full more often, and with more entries. Furthermore, splits are more frequent because nodes tend to be fuller. The curves are rough because node eliminations occur randomly and infrequently; there were too few in our tests to smooth out the variations.

Figures 4.4 and 4.5 show that the search performance of the index is very

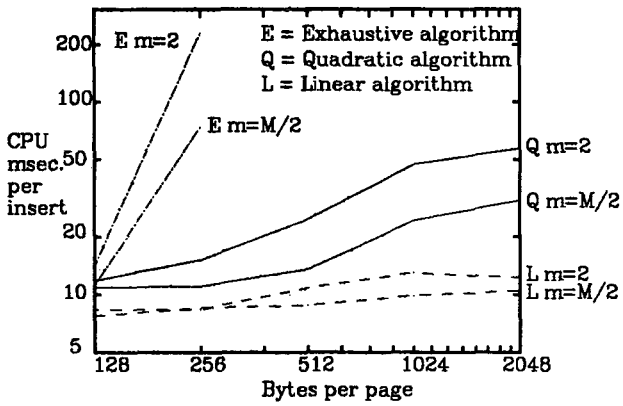


Figure 4.2
CPU cost of inserting records.

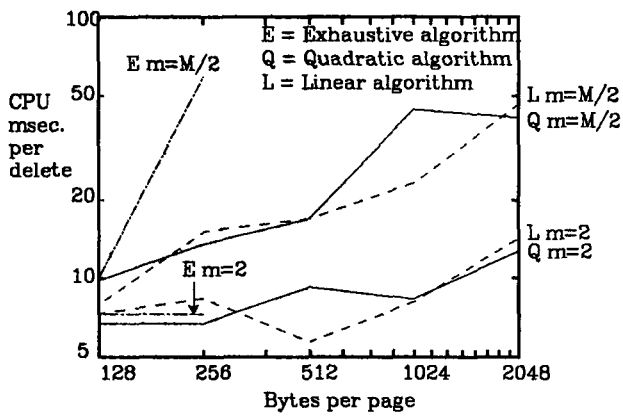


Figure 4.3
CPU cost of deleting records.

insensitive to the use of different node split algorithms and fill requirements. The exhaustive algorithm produces a slightly better index structure, resulting in fewer pages touched and less CPU cost, but most combinations of algorithm and fill requirement come within 10% of the best. All algorithms provide reasonable performance.

Figure 4.6 shows the storage space occupied by the index tree as a function of algorithm, fill criterion and page size. Generally the results bear out our expectation that stricter node fill criteria produce smaller indexes. The least dense index consumes about 50% more space than the most dense, but all results for 1/2-full and 1/3-full (not shown) are within 15% of each other.

A second series of tests measured R-tree performance as a function of the amount of data in the index. The same sequence of test operations as before was

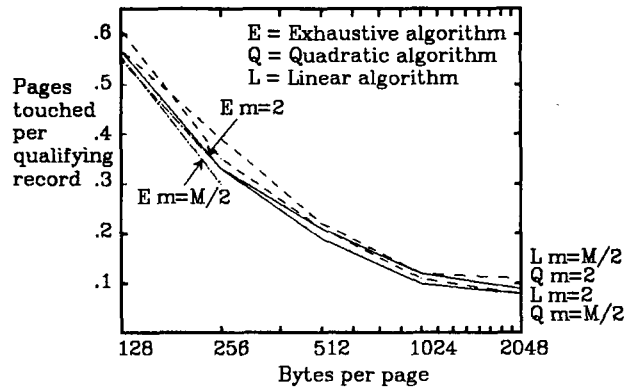


Figure 4.4
Search performance: Pages touched.

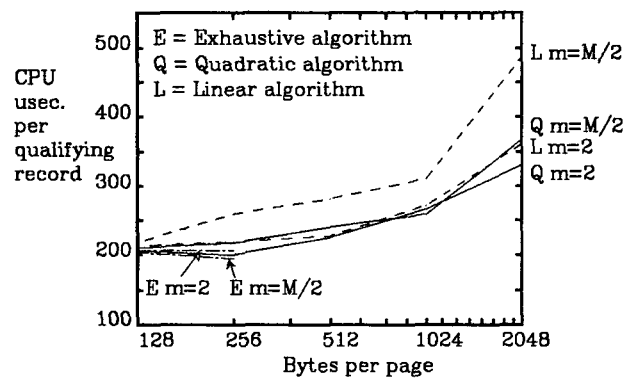


Figure 4.5
Search performance: CPU cost.

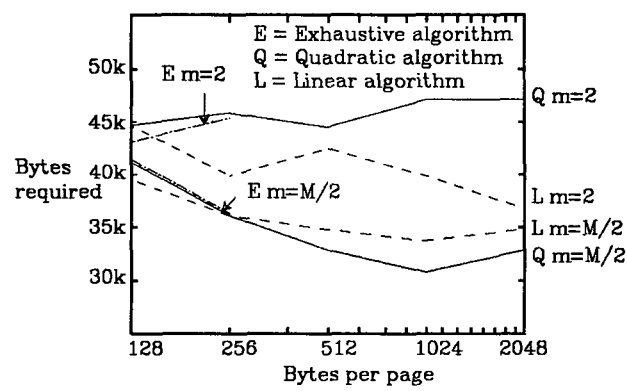


Figure 4.6
Space efficiency.

run on samples containing 1057, 2238, 3295, and 4559 rectangles. The first sample contained layout data from the circuit cell CENTRAL used earlier, and the second consisted of layout from a similar but larger cell containing 2238 rectangles. The third sample was made by using both

CENTRAL and the larger cell, with the two cells effectively placed on top of each other. Three cells were combined to make up the last sample. Because the samples were composed in different ways using varying data, performance results do not scale perfectly and some unevenness was to be expected.

Two combinations of split algorithm and node fill requirement were chosen for the tests: the linear algorithm with $m=2$, and the quadratic algorithm with $m=M/3$, both with a page size of 1024 bytes ($M=50$).

Figure 4.7 shows the results of tests to determine how insert and delete performance is affected by tree size. Both test configurations produced trees with two levels for 1057 records and three levels for the other sample sizes. The figure shows that the cost of inserts with the quadratic algorithm is nearly constant except where the tree increases in height. There the curve shows a definite jump because of the increase in the number of levels where a split can occur. The linear algorithm shows no jump, indicating again that linear node splits account for only a small part of the cost of inserts.

No node splits occurred during the deletion tests with the linear configuration, because of the relaxed node fill requirement and the small number of data items. As a result the curve shows only a small jump where the number of tree levels increases. Deletion with the quadratic configuration produced only 1 to 6 node splits, and the resulting curve is very rough. When allowance is made for variations due to the small sample size, the tests show that insert and delete cost is independent of tree width but is affected by tree height, which grows slowly with the number of data items.

Figures 4.8 and 4.9 confirm that the two configurations have nearly the same search performance. Each search retrieved between 3% and 6% of the data. The downward trend of the curves is to be expected, because the cost of processing higher tree nodes becomes less significant as the amount of data retrieved in each search increases. The increase in the number of tree levels kept the cost from dropping between the first and second data points. The low CPU cost per qualifying record, less than 150 microseconds for larger amounts of data, shows that the index is quite effective in narrowing searches to small subtrees.

The straight lines in Figure 4.10 reflect the fact that almost all the space in an R-tree index is used for leaf nodes, whose number varies linearly with the amount of data. For the Linear-2 test configuration the total space occupied by the R-tree was about 40 bytes per data item, compared to 20 bytes per item for the index records alone. The corresponding figure for the Quadratic-1/3 configuration was 33 bytes per item.

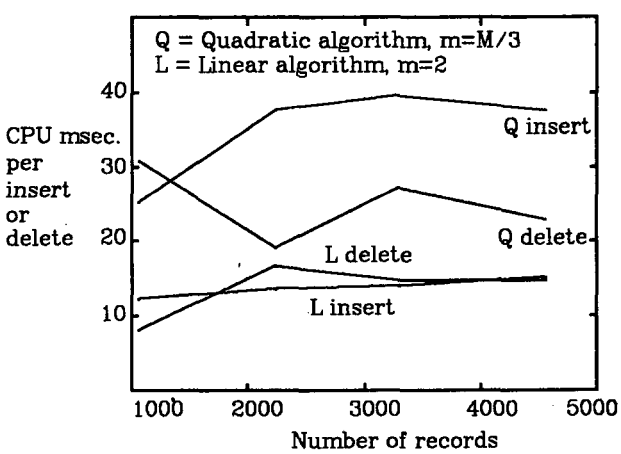


Figure 4.7
CPU cost of inserts and deletes vs. amount of data.

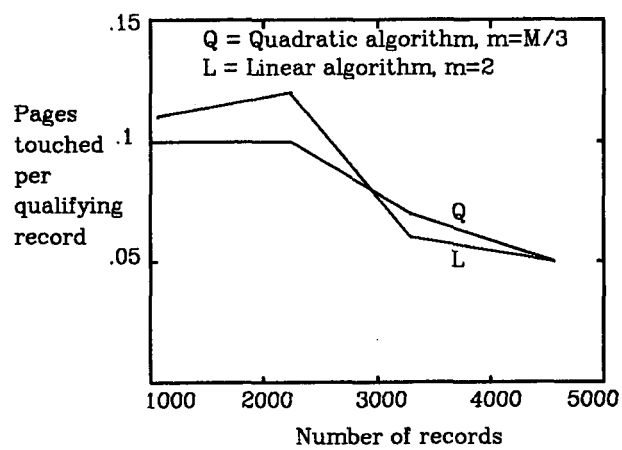


Figure 4.8
Search performance vs. amount of data: Pages touched

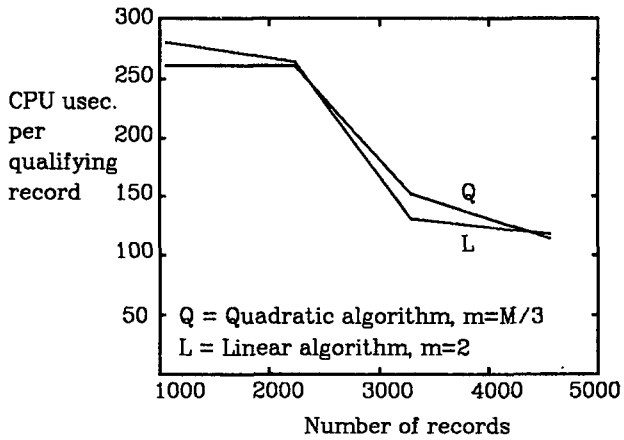


Figure 4.9
Search performance vs. amount of data: CPU cost

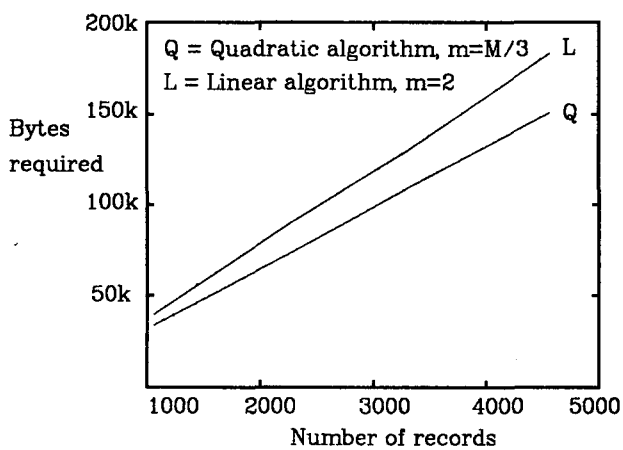


Figure 4.10
Space required for R-tree vs. amount of data.

5. Conclusions

The R-tree structure has been shown to be useful for indexing spatial data objects that have non-zero size. Nodes corresponding to disk pages of reasonable size (e.g. 1024 bytes) have values of M that produce good performance. With smaller nodes the structure should also be effective as a main-memory index; CPU performance would be comparable but there would be no I/O cost.

The linear node-split algorithm proved to be as good as more expensive techniques. It was fast, and the slightly worse quality of the splits did not affect search performance noticeably.

Preliminary investigation indicates that R-trees would be easy to add to any relational database system that supported conventional access methods, (e.g. INGRES [9], System-R [1]). Moreover, the new structure would work especially well in conjunction with abstract data types and abstract indexes [14] to streamline the handling of spatial data.

6. References

1. M. Astrahan, et al., System R: Relational Approach to Database Management, *ACM Transactions on Database Systems 1*, 2 (June 1976), 97-137.
2. R. Bayer and E. McCreight, Organization and Maintenance of Large Ordered Indices, *Proc. 1970 ACM-SIGFIDET Workshop on Data Description and Access*, Houston, Texas, Nov. 1970, 107-141.
3. J. L. Bentley, Multidimensional Binary Search Trees Used for Associative Searching, *Communications of the ACM 18*, 9 (September 1975), 509-517.
4. J. L. Bentley, D. F. Stanat and E. H. Williams, Jr., The complexity of fixed-radius near neighbor searching, *Inf. Proc. Lett. 6*, 6 (December 1977), 209-212.
5. J. L. Bentley and J. H. Friedman, Data Structures for Range Searching, *Computing Surveys 11*, 4 (December 1979), 397-409.
6. D. Comer, The Ubiquitous B-tree, *Computing Surveys 11*, 2 (1979), 121-138.
7. R. A. Finkel and J. L. Bentley, Quad Trees - A Data Structure for Retrieval on Composite Keys, *Acta Informatica 4*, (1974), 1-9.
8. A. Guttman and M. Stonebraker, Using a Relational Database Management System for Computer Aided Design Data, *IEEE Database Engineering 5*, 2 (June 1982).
9. G. Held, M. Stonebraker and E. Wong, INGRES - A Relational Data Base System, *Proc. AFIPS 1975 NCC 44*, (1975), 409-416.
10. K. Hinrichs and J. Nievergelt, The Grid File: A Data Structure Designed to Support Proximity Queries on Spatial Objects, Nr. 54, Institut fur

Informatik, Eidgenossische Technische Hochschule, Zurich, July 1983.

11. M. G. H. Katevenis, R. W. Sherburne, D. A. Patterson and C. H. Séquin, The RISC II Micro-Architecture, *Proc. VLSI 83 Conference*, Trondheim, Norway, August 1983.

12. J. K. Ousterhout, Corner Stitching: A Data Structuring Technique for VLSI Layout Tools, Computer Science Report Computer Science Dept. 82/114, University of California, Berkeley, 1982.

13. J. T. Robinson, The K-D-B Tree: A Search Structure for Large Multidimensional Dynamic Indexes, *ACM-SIGMOD Conference Proc.*, April 1981, 10-18.

14. M. Stonebraker, B. Rubenstein and A. Guttman, Application of Abstract Data Types and Abstract Indices to CAD Data Bases, Memorandum No. UCB/ERL M83/3, Electronics Research Laboratory, University of California, Berkeley, January 1983.

15. K. C. Wong and M. Edelberg, Interval Hierarchies and Their Application to Predicate Files, *ACM Transactions on Database Systems 2*, 3 (September 1977), 223-232.

16. G. Yuval, Finding Near Neighbors in k-dimensional Space, *Inf. Proc. Lett. 3*, 4 (March 1975), 113-114.